

DESIGN AND ANALYSIS ALGORITHMS

LECTURE 3

Recursion

Reference link and books

Prof. Xukai Zou, Computer Science Dept.,
Indiana University Purdue University Indianapolis
<http://cs.iupui.edu/~xzou/>

Ian Parberry and W. Gasarch, *Problems on Algorithms*,
ebook, 2002

Manuel Rubio-Sánchez, *Introduction to recursive programming*, CRC Press, 2018.

Review the Previous Lesson

- The analysis framework
 - Input data size and Basic operation
 - Orders of growth; Best/Worst/Average cases
 - Analyze the complexity of algorithm
 - Asymptotic notations
 - Establishing rate of growth relation
 - Prove the correctness of algorithm
 - Inductive for recursive algorithm
 - Loop Invariant for iterative algorithm
-

Lecture outline

- ❑ Introduction
- ❑ Format of a recursive algorithm
- ❑ Analysis of recursive algorithm
- ❑ Iterative instead recursive
- ❑ Exercises

Introduction

-
- ✓ What is recursion?
 - ✓ Recursive problem examples
-

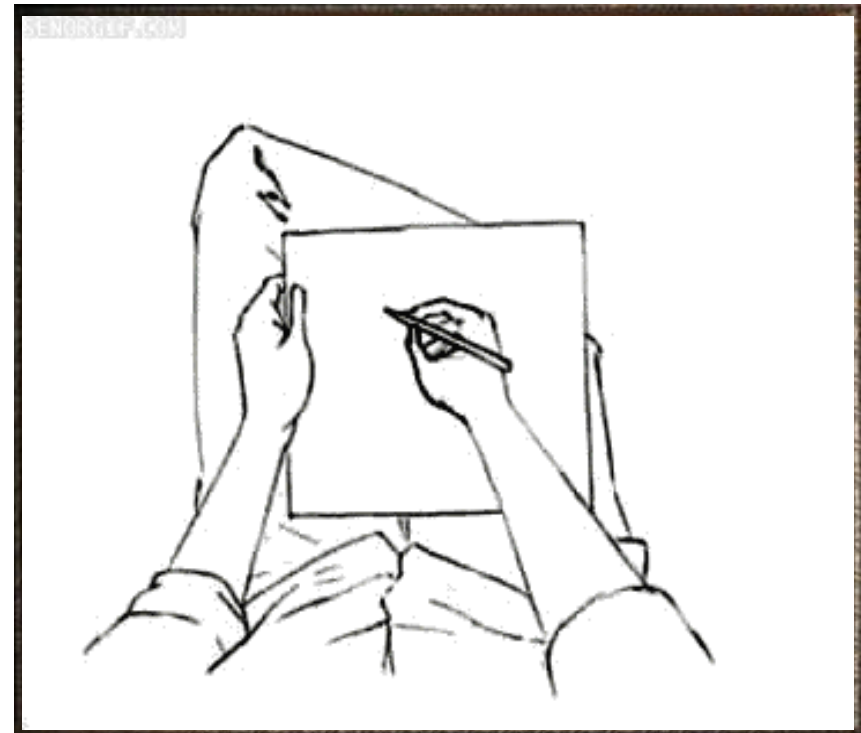
What is recursion?

- ❑ Recursive images



Matryoshka dolls

Manuel's book



<https://gifer.com/en/gifs/recursion>

What is recursion?

□ Recursion

- Recursion is encountered in some wonders of nature:
 - Tree and branch of a tree;
 - Blood vessels or rivers branching patterns;
 - Mountain ranges, clouds;...
 - A broad concept used in diverse disciplines: mathematics, bioinformatics, linguistics, and art...
 - In computer programming, recursion is a powerful strategy that allows us to design simple, succinct, and elegant algorithms.
(thuật toán đơn giản, gọn gàng, tinh tế)
-

What is recursion?

❑ Recursive algorithm

- A problem can be solved by using solutions of smaller versions of the same problem, can be solved by a recursive algorithm.
- Recursive algorithm (T) is an algorithm which calls itself (T') with "smaller (or simpler)" input values.

❑ Finiteness of recursive algorithm

- T' must "smaller (or simpler)" than T.
- The smallest (or simplest) version is solved.

❑ Example: Compute $S(n) = n!$ with input n

$$S(n)=n! \leftarrow n*S(n-1) \leftarrow (n-1)*S(n-2) \leftarrow \dots \leftarrow 2*S(1) \leftarrow S(1) = 1$$

What is recursion?

- ❑ Problems that can be solved by recursion:
 - **Parameter dependent** problems;
 - In some certain values of the parameter, the problem has solution (**base case**).
 - In the **general case**, the problem can be converted to a similar form with new parameters and after finite steps it lead to base case.
-

Recursive problem examples

□ Fibonacci Sequence: Find the n^{th}

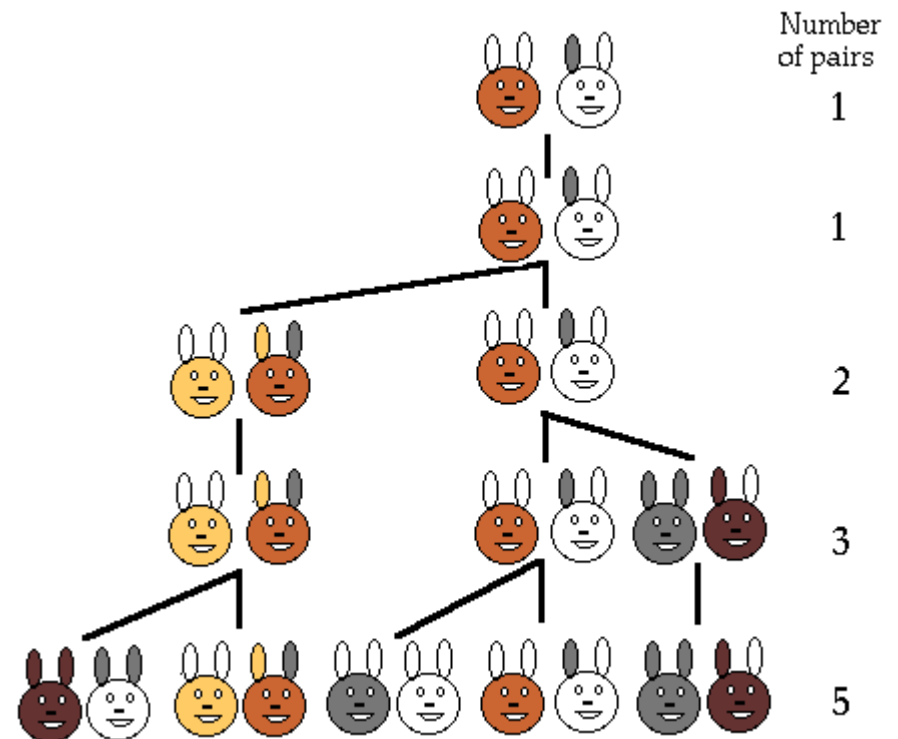
Algorithm calculate $\text{Fib}(n)$;

- Base case :

$n=1$ or $n=2$ then $\text{Fib}(n)=1$

- General case ($n>2$):

$\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$



Xem thêm về số Fibonacci:

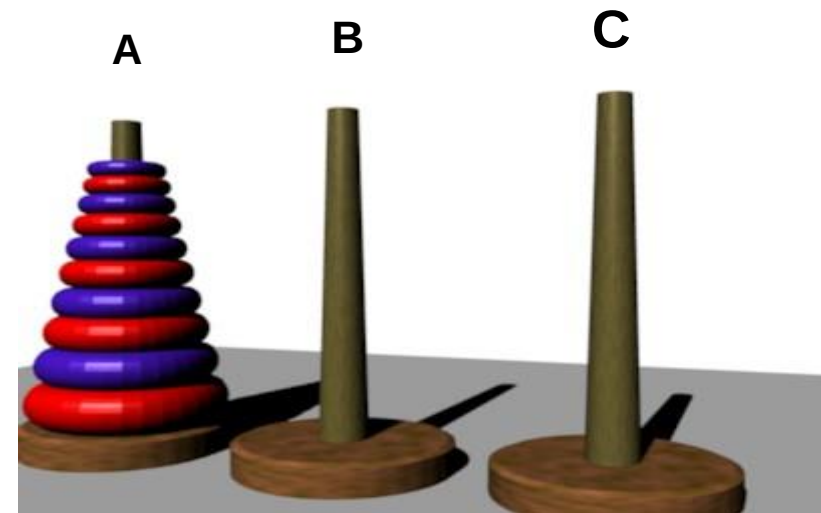
<http://www.maths.surrey.ac.uk/hosted-sites/R.Knott/Fibonacci/fibnat.html>

Recursive problem examples

- Tower of Hanoi Problem: a game require to move the n -disks tower from the rod A to rod B and use rod C as auxiliary.

Algorithm MoveTower(n, A, B, C);

- Base case: $n=1$ Move1Disk(A, B);
- General case ($n>1$):
 MoveTower(n, A, C, B);
 Move1Disk(A, B);
 MoveTower($n-1, C, B, A$);

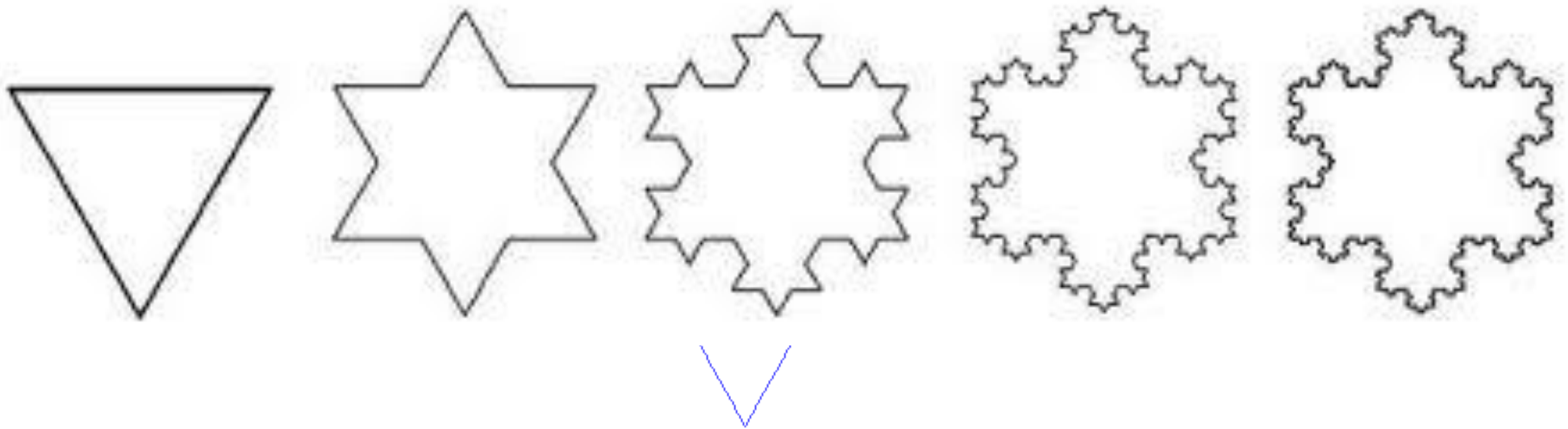


Where: Move1Disk(A, B) is move the top disk from $A \rightarrow B$

Recursive problem examples

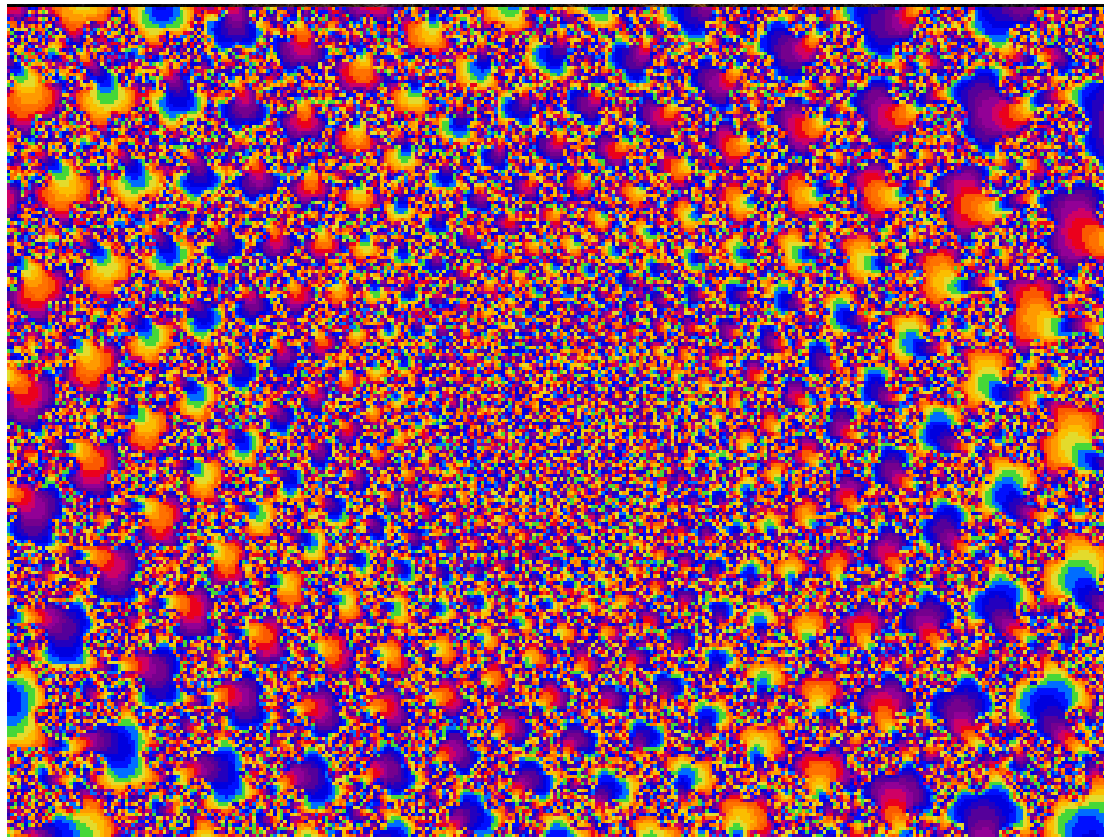
□ Recursion and Fractal

Fractal: can be understood as a geometric object whose structure simple is repeated at different scales (positions, colors...).



Recursive problem examples

□ Recursion and Fractal



<https://gifer.com/en/gifs/fractal>

<https://www.youtube.com/watch?v=u1pwtSBTnPU>

Recursive Algorithm Diagram

Recursive Algorithm Diagram

□ Diagram:

```
Recursive_Algorithm( $p_n$ )  $\equiv$  // $p_n$  - parameter(s)  
  if ( $p_n=p_0$ )    //base case  
    <statements;>  
  else    //general case  
    <statements;>  
    Recursive_Algorithm( $p_{n-1}$ );  
    <statements;>  
  endif  
End.
```

$p_n \leftarrow p_{n-1} \leftarrow p_{n-2} \leftarrow \dots \leftarrow p_0$

Recursive Algorithm Diagram

□ Examples:

■ Calculate n!

```
S(n):int ≡ //Recursive function return n! value
    if (n==1)
        S = 1;
    else
        S = n * S(n-1)
End.
```


Recursive Algorithm Diagram

□ Examples:

■ Calculate Fib(n)

```
Fib(n):int ≡ //Recursive function return nth fib number  
    if (n==1 || n==2)  
        Fib = 1;  
    else  
        Fib = Fib(n-1)+Fib(n-2);  
End.
```

Recursive Algorithm Diagram

□ Examples: Tower of Hanoi

```
MoveTower(n,A,B,C) ≡ //function move tower n-disks from A to B  
    if (n=1)  
        Move1Disk(A,B);  
    else{  
        MoveTower(n-1,A,C,B);  
        Move1Disk(A,B);  
        MoveTower(n-1,C,B,A);  
    }  
End.
```

```
Move1Disk(A,B) ≡ //function move the disk on top of A to B  
    print(A,"→",B)  
End.
```

Analyze Recursive Algorithm

-
- ✓ Correctness of recursive algorithm
 - ✓ Complexity of recursive algorithm
-

Correctness of recursive algorithm

- Prove recursive algorithm correctness by **induction**:
 - Prove by induction on the “size” of the problem.
 - **Base case** of recursion is base of induction.
 - **Inductive assumption**: Assume that the recursive call work correctly with input size $f(n)$.
 - **General case**: Use the assumption to prove that the current call works correctly with input size n .
- Where: $f(n)$ is function make n smaller (or simpler) in each step.
-

Complexity of recursive algorithm

□ Deriving recurrence relations :

- Recurrence relations for assessing the running time.
 - Let $T(n)$ is running time of the algorithm with input size n .
 - Figure out what “ n ”, the problem size, is.
 - What value of n is base case, suppose n_0 .
 - Figure out what $T(n_0)$ is, suppose const c .
 - $T(n)$ in general case is sum of time of recursive calls of a subproblems size $f(n)$ plus time of the non-recursive work.
-

Complexity of recursive algorithm

❑ Deriving recurrence relations:

The diagram shows a recurrence relation for $T(n)$ with several annotations explaining its components:

- Running time for base**: Points to the constant c in the base case.
- Base of recursion**: Points to n_0 in the base case.
- Number of times recursive calls**: Points to the coefficient a in the recursive case.
- Size of problem solved by recursive call**: Points to $f(n)$ in the recursive case.
- All other processing**: Points to $g(n)$ in the recursive case.

$$T(n) = \begin{cases} c & \text{if } n = n_0 \\ a.T(f(n)) + g(n) & \text{otherwise} \end{cases}$$

Complexity of recursive algorithm

□ Examples of deriving recurrence relations:

- $S(n)$ algorithm

$$T_S(n) = \begin{cases} 1 & \text{if } n = 1 \\ T_S(n-1) + 1 & \text{if } n > 1 \end{cases}$$

- $Fib(n)$ algorithm

$$T_{Fib}(n) = \begin{cases} 1 & \text{if } n = 1, 2 \\ T_{Fib}(n-1) + T_{Fib}(n-2) + 1 & \text{if } n > 2 \end{cases}$$

- $MoveTower(n, A, B, C)$ algorithm

$$T_{Move}(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2T_{Move}(n-1) + 1 & \text{if } n > 1 \end{cases}$$

Complexity of recursive algorithm

□ Solving recurrence relations:

- Use repeated substitution

$$T_S(n) = \begin{cases} 1 & \text{if } n = 1 \\ T_S(n-1) + 1 & \text{if } n > 1 \end{cases}$$

$$\begin{aligned} T_S(n) &= T_S(n-1) + 1 = T_S(n-2) + 1 + 1 \\ &= T_S(n-3) + 1 + 1 + 1 \\ &= \dots \\ &= T_S(n - (n-1)) + \underbrace{1 + \dots + 1}_{n-1 \text{ times}} \\ &= 1 + 1 + \dots + 1 = n \end{aligned}$$

$$\Rightarrow T_S(n) = O(n)$$

Complexity of recursive algorithm

□ Solving recurrence relations:

- A general theorem: If n is a power of b , the solution to the recurrence

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ aT(n/b) + n^d & \text{if } n > 1, a \geq 1, b > 1, d \geq 0 \end{cases}$$

is

$$T(n) = \begin{cases} O(n^d) & \text{if } a < b^d \\ O(n^d \log n) & \text{if } a = b^d \\ O(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Complexity of recursive algorithm

□ Proof of general theorem:

$$\begin{aligned} T(n) &= a^{\log_b n} T(n/b^{\log_b n}) + \dots + a^2 (n/b^2)^d + a(n/b)^d + n^d \\ &= n^{\log_b a} + n^d \sum_{j=0}^{\log_b n - 1} (a/b^d)^j \end{aligned}$$

- In case: $a < b^d$

$$T(n) < n^{\log_b a} + n^d \frac{1}{1 - (a/b^d)} = O(n^d)$$

- In case: $a = b^d$

$$T(n) = n^{\log_b a} + n^d \log_b n = O(n^d \log_b n)$$

- In case: $a > b^d$

$$T(n) < n^{\log_b a} + n^d (a/b^d)^{\log_b n} \frac{1}{(a/b^d) - 1} = n^{\log_b a} \left(1 + \frac{1}{(a/b^d) - 1}\right) = O(n^{\log_b a})$$

Complexity of recursive algorithm

□ Applying general theorem:

▪ Example 1

$$T(n) = 2T(n/2) + n^2$$

$$a = 2, b = 2, d = 2, a < b^d \rightarrow T(n) = O(n^d) = O(n^2)$$

▪ Example 2

$$T(n) = 2T(n/2) + n$$

$$a = 2, b = 2, d = 1, a = b^d \rightarrow T(n) = O(n^d \log_b n) = O(n \log n)$$

▪ Example 3

$$T(n) = 2T(n/2) + 1$$

$$a = 2, b = 2, d = 0, a > b^d \rightarrow T(n) = O(n^{\log_b a}) = O(n)$$

Complexity of recursive algorithm

□ Applying general theorem:

▪ Example 4

$$T(n) = 4T(n/2) + n^3$$

$$a = 4, b = 2, d = 3, a < b^d \rightarrow T(n) = O(n^d) = O(n^3)$$

▪ Example 5

$$T(n) = 4T(n/2) + n^2$$

$$a = 4, b = 2, d = 1, a = b^d \rightarrow T(n) = O(n^d \log n) = O(n^2 \log n)$$

▪ Example 6

$$T(n) = 4T(n/2) + n$$

$$a = 4, b = 2, d = 1, a > b^d \rightarrow T(n) = O(n^{\log_b a}) = O(n^2)$$

Recursion Elimination - Khử đệ quy

-
- ✓ Khái niệm khử đệ quy
 - ✓ Khử đệ quy một số dạng thường gặp
-

Khái niệm khử đệ quy

□ Khái niệm

Ưu điểm của các giải thuật đệ quy: Ngắn gọn, trong sáng.

Nhược điểm: Lời gọi thực hiện phức tạp hơn, đôi khi khó hình dung.

Lí do: Mỗi lần gọi đệ quy, một tập các giá trị cục bộ được tạo ra để lưu các giá trị trung gian phục vụ cho các lượt tính ngược lại.

Thay đổi: Chủ động quản lí các giá trị trung gian bằng ngăn xếp, thay lời gọi đệ quy bằng vòng lặp $\Rightarrow$ **thuật toán khử đệ quy**.

Khử đệ quy một thuật toán đệ quy là thay thế thuật toán đệ quy bằng một thuật toán **tương đương** nhưng bỏ đi các lời gọi đệ quy và thay vào đó bằng việc sử dụng các vòng lặp cùng với sự hỗ trợ của ngăn xếp dữ liệu (*stack*)

Khử đệ quy một số dạng thường gặp

□ Dạng ED[AP]

```
P (x) ≡  
    if (E (x) )  
        D (x) ;  
    else  
        A (x) ;  
        P (f (x) ) ;  
    endif  
End.
```


*Triển khai
chi tiết*

E (x) : điều kiện suy biến
D (x) : lời giải trong trường hợp suy biến
A (x) : thao tác trước lời gọi đệ quy
f (x) : hàm biến đổi tham số lời gọi đệ quy

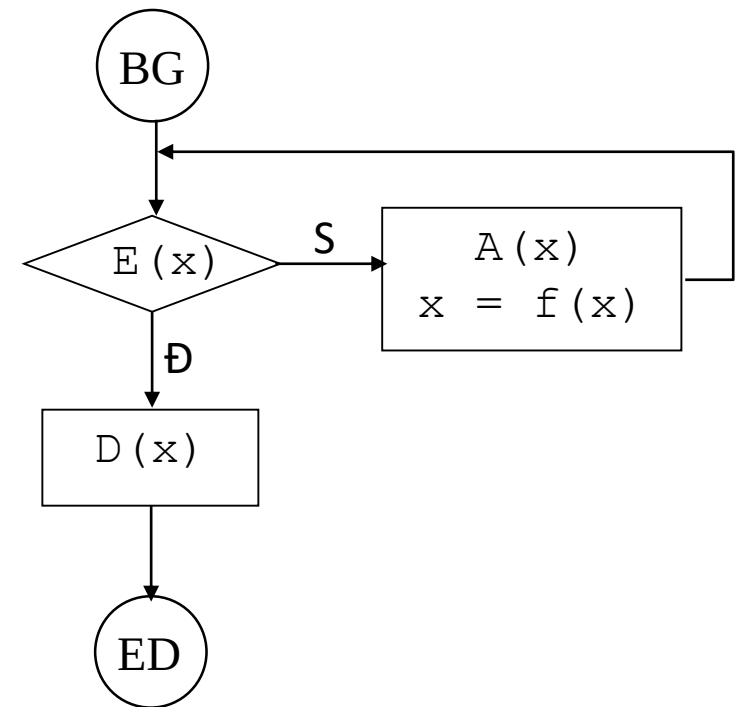
```
P (x) ≡  
    if E (x)  
        D (x) ;  
    else  
        A (x) ;  
        x = f (x) ;  
        if (E (x) )  
            D (x) ;  
        else  
            A (x) ;  
            x = f (x) ;  
            ...  
        endif  
    endif  
End.
```

Khử đệ quy một số dạng thường gặp

□ Dạng ED[AP]

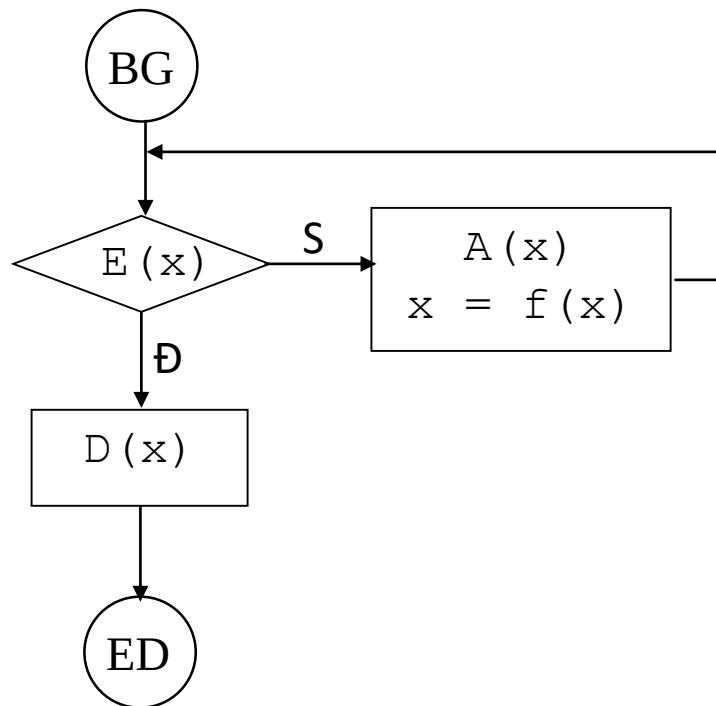
```
P (x)  ≡  
  if E (x)  
    D (x) ;  
  else  
    A (x) ;  
    x = f (x) ;  
    if (E (x))  
      D (x) ;  
    else  
      A (x) ;  
      x = f (x) ;  
    ...  
  endif  
endif  
End.
```

→
*Sơ đồ
thực hiện*



Khử đệ quy một số dạng thường gặp

□ Dạng ED[AP]



→
*Sử dụng
vòng lặp*

$P(x) \equiv$
while not $E(x)$ **do**
 $A(x)$;
 $x := f(x)$;
endwhile
 $D(x)$;
End.

Khử đệ quy một số dạng thường gặp

□ Dạng ED[AP]

Ví dụ: phân tích số nguyên thành các thừa số nguyên tố, in theo thứ tự tăng dần.

```
PhanTich(n) ≡  
    d = 2;  
    if (n=1)  
        exit  
    else  
        while (n mod d <> 0) do  
            d := d+1  
            write(d);  
            PhanTich(n div d);  
        endif  
End.
```

Các thành phần của giải thuật đệ quy dạng ED[AP] của thuật toán PhanTich

```
Tham số (x) ≡ (n)  
E(x) ≡ n=1  
D(x) ≡ exit  
A(x) ≡ d = 2;  
        while (n mod d <> 0) do  
            d := d+1  
            write(d);  
f(x) ≡ f(n) = n div d
```

Khử đệ quy một số dạng thường gặp

□ Dạng ED[AP]

```
P(x) ≡  
    while not E(x) do  
        A(x);  
        x := f(x);  
    endwhile  
D(x);  
End.
```

```
PhanTich(n)≡  
    d = 2;  
    while (n>1) do  
        while (n mod d <> 0) do  
            d := d+1;  
        endwhile;  
        write(d);  
        n = n div d;  
    endwhile;  
    exit;  
End.
```

Khử đệ quy một số dạng thường gặp

□ Dạng ED[APB]

```
P (x) ≡  
    if (E (x))  
        D (x) ;  
    else  
        A (x) ;  
        P (f (x)) ;  
        B (x) ;  
    endif  
End.
```


*Triển khai
chi tiết*

$E(x)$: điều kiện suy biến
 $D(x)$: lời giải trong trường hợp suy biến
 $A(x), B(x)$: thao tác trước lời gọi đệ quy
 $f(x)$: hàm biến đổi tham số lời gọi đệ quy

```
P (x) ≡  
    if E (x)  
        D (x) ;  
    else  
        A (x) ;  
        x = f (x) ;      // f1  
        if (E (x))  
            D (x) ;  
        else  
            A (x) ;  
            x = f (x) ;  // f2  
            ...  
        endif  
        B (x) // x=f2 (x)  
    endif  
B (x) // x=f1 (x)  
End.
```

Khử đệ quy một số dạng thường gặp

□ Dạng ED[APB]

```
P(x) ≡
  if E(x)
    D(x);
  else
    A(x);
    x = f(x);    // f1
    if (E(x))
      D(x);
    else
      A(x);
      x = f(x);  // f2
      ...
    endif
  B(x) // x=f2(x)
endif
B(x) // x=f1(x)
End.
```

P kết thúc sau k+1 lần gọi:

- $E(f^k(x)) = \text{true}$ với

$$f^k(x) = f(f \dots (f(x)) \dots)$$

$$f^0(x) = x$$

- Sau khi thực hiện $D(f^k(x))$ sẽ phải thực hiện $B(f^k(x))$

- Tiếp đó sẽ thực hiện $B(f^{k-1}(x)) \dots$ và cuối cùng là $B(x)$.

Tóm lại, giá trị của x mỗi lần biến đổi $x=f(x)$ được cất giữ thành chồng (stack) rồi dỡ ra theo chiều từ trên xuống để thực hiện $B(x)$

Khử đệ quy một số dạng thường gặp

□ Dạng ED[APB]

Sử dụng ngăn xếp S, kiểu phần tử phù hợp với x và hai vòng lặp để xây dựng thuật toán khử đệ quy:

```
P (x) ≡
    Start (S );           //Khởi tạo ngăn xếp S
    while not E(x) do // Vòng lặp khử lời gọi đệ quy
        A(x);
        Push(x,S);       //Cất x vào ngăn xếp
        x := f(x);
    endwhile;
    D(x)
    while not Is_Empty(S) do //Vòng lặp gỡ ngăn xếp
        Pick(S,x);       //Lấy x từ ngăn xếp
        B(x)
    endwhile;
End.
```

Khử đệ quy một số dạng thường gặp

□ Dạng ED[APB]

Ví dụ: Đổi số thập phân sang nhị phân

```
Int2Bin(n)≡  
  if (n>0)  
    Int2Bin(n div 2);  
    Write(n mod 2);  
  endif;  
End.
```

Các thành phần của giải thuật đệ quy dạng ED[APB] của thuật toán `Int2Bin`

Tham số $(x) \equiv (n)$

$E(x) \equiv n=0$

$D(x) \equiv \emptyset$

$A(x) \equiv \emptyset$

$f(x) \equiv f(n) = n \text{ div } 2$

$B(x) \equiv \text{write}(n \text{ mod } 2)$

Khử đệ quy một số dạng thường gặp

□ Dạng ED[APB]

Ví dụ: Đổi số thập phân sang nhị phân

```
Int2Bin(n)≡  
    if (n>0)  
        Int2Bin(n div 2);  
        Write(n mod 2);  
    endif;  
End.
```

Sử dụng mảng làm ngăn xếp

```
int S[50];  
int st;
```

```
IntToBin(n)  ≡  
    st = 0; //Start(S)  
    while (n>0) do  
        st++;  
        S[st]=n mod 2; //Push(S)  
        n = n div 2;  
    endwhile;  
    while (st<>0) do  
        write(s[st]); //Pick(S)  
        st--;  
    endwhile;  
End.
```

Dùng chuỗi kí tự làm ngăn xếp sẽ cho chương trình ngắn gọn hơn

Khử đệ quy một số dạng thường gặp

□ Dạng ED[APBP]

```
P (x)  ≡  
    if  (E (x) )  
        D (x) ;  
    else  
        A (x) ;  
        P (f (x) ) ;  
        B (x) ;  
        P (g (x) ) ;  
    endif  
End.
```

E (x) : điều kiện suy biến

D (x) : lời giải trong trường hợp suy biến

A (x) , B (x) : thao tác ngoài lời gọi đệ quy

f (x) , g (x) : hàm biến đổi tham số của
hai lời gọi đệ quy

Khử đệ quy một số dạng thường gặp

□ Dạng ED[APBP]

```
P (x) ≡  
    if (E (x) )  
        D (x) ;  
    else  
        A (x) ;  
        P (f (x) ) ;  
        B (x) ;  
        P (g (x) ) ;  
    endif  
End.
```

```
P (x) ≡  
    Start (S) ;  
    Push ( (x, 1) , S) ;  
    repeat  
        while not E(x) do  
            A (x) ;  
            Push ( (x, 2) , S) ;  
            x := f (x) ;  
        endwhile;  
        D (x) ;  
        Pick (S, (x, m) ) ;  
        if (m=2)  
            B (x) ; x= g (x) ;  
        endif;  
    until (m=1) ;  
End.
```

Khử đệ quy một số dạng thường gặp

□ Dạng ED[APBP]

Ví dụ: Bài toán tháp Hà Nội

```
MoveTower (n, A, B, C)  $\equiv$   
  if (n=1)  
    Move1Disk (A, B) ;  
  else  
    MoveTower (n-1, A, C, B) ;  
    Move1Disk (A, B) ;  
    MoveTower (n-1, C, B, A) ;  
  endif  
End.
```

```
P (x)  $\equiv$  MoveTower (n, A, B, C)  
E (x)  $\equiv$  (n=1)  
D (x)  $\equiv$  Move1Disk (A, B)  
A (x)  $\equiv \emptyset$   
f (x)  $\equiv$  f (n, A, B, C) = (n-1, A, C, B)  
do đó :  $n \rightarrow n-1, A \rightarrow A,$   
           $B \rightarrow C, C \rightarrow B$   
B (x)  $\equiv$  Move1Disk (A, B)  
g (x)  $\equiv$  g (n, A, B, C) = (n-1, C, B, A)  
do đó :  $n \rightarrow n-1, A \rightarrow C,$   
           $B \rightarrow B, C \rightarrow A$ 
```

Khử đệ quy một số dạng thường gặp

□ Dạng ED[APBP]

Ví dụ: Bài toán tháp Hà Nội

```
P(x)  ≡ MoveTower(n,A,B,C)
E(x)  ≡      (n=1)
D(x)  ≡ Move1Disk(A,B)
A(x)  ≡ ∅
f(x)  ≡ f(n,A,B,C)=(n-1,A,C,B)
      do đó : n→n-1, A→A,
              B→C, C→B
B(x)  ≡ Move1Disk(A,B)
g(x)  ≡ g(n,A,B,C)=(n-1,C,B,A)
      do đó : n→n-1, A→C,
              B→B, C→A
```

```
MoveTower(n,A,B,C)  ≡
  Start(S);
  Push((n,A,B,C,1),S);
  repeat
    while (n>1) do
      Push((n,A,B,C,2),S);
      n--;
      Swap(B,C);
    endwhile;
    Move1Disk(A,B);
    Pick(S,(n,A,B,C,m))
    if (m=2)
      Move1Disk(A,B);
      n--;
      Swap(A,C);
    endif
  until (m=1);
End.
```

Exercises

- ❑ Solving recursive relations.
 - ❑ Recursive programming.
 - ❑ Design and analysis recursive algorithm.
 - ❑ More detail in [Hw3_Recursion.doc](#)
-